

# Informe Técnico Exhaustivo y Arquitectura de Plataforma

Auditoría técnica integral del ecosistema **PrismaLab / Plataforma CCV 3.0**: revisión estructural de código, modelo relacional PostgreSQL (Supabase), políticas de seguridad RLS jerárquicas, motor de tareas con dependencias y pipeline operativo de producción virtual.

## Next 14

APP ROUTER & HYBRID  
UI

## 15 Tablas

MODELO RELACIONAL  
DDL

## 6 Niveles

JERARQUÍA RBAC &  
RLS

## 100% Cascada

DESBLOQUEO DE  
TAREAS

### ORGANIZACIÓN / ENTIDAD

Centro de Educación Virtual (CCV)

### PERFIL AUDITOR

Arquitecto de Software Senior & Auditor Full-Stack

### FECHA DEL INFORME

Septiembre 2026

### PLATAFORMA & VERSIÓN

PrismaLab CCV v3.0.0 (Production-Ready)

### STACK PRINCIPAL

Next.js 14, React 18, TypeScript, Tailwind CSS, Supabase

### ESTADO DE VALIDACIÓN

✓ Build Verificado & Git Sync en Main

# 1 Estructura General y Arquitectura de Directorios

La plataforma **PrismaLab (CCV 3.0)** está estructurada como una solución web full-stack moderna construida sobre **Next.js 14** con la convención del *App Router*. Su arquitectura implementa un enfoque **Feature-Based & Modular** combinado con **Separación Estricta de UI y Lógica de Negocio**, permitiendo que la plataforma escale sin incurrir en acoplamientos rígidos entre componentes visuales y la capa de datos.

## 1.1 Árbol de Directorios del Código Fuente (`/src`)

src/

app/

— App Router de Next.js. Define rutas principales (`page.tsx`), layouts (`layout.tsx`), estilos globales (`globals.css`) y endpoints API protegidos (`/api/admin/users`).

components/

— Librería modular de componentes de interfaz categorizada por 13 dominios de negocio:

academic/

— Modales y árboles de Facultades, Programas, Cursos, Proyectos Especiales y progreso del curso.

admin/

— Panel de Control RBAC, Matriz de Roles, Tarifas, Aprobación de Usuarios, y Catálogo de Plantilla Cursos.

tasks/

— Tarjetas de tareas (`TaskCard`), detalle modal enriquecido (`TaskDetailModal`) y creación dinámica.

kanban/

— Tablero Kanban con Drag & Drop nativo y persistencia de estados en tiempo real.

calendar/

— Calendario operativo de trabajo con visualización cronológica mensual y semanal.

planner/

— Parrilla de Publicaciones CCV y Calendario Editorial con segmentación multicanal.

productivity/

— Barra flotante global de cronómetro (`GlobalTimerBar`) y seguimiento de tiempos en vivo.

dashboard/

— Módulo analítico, tarjetas de KPI ejecutivos y gráficos interactivos con Recharts.

auth/

— Pantalla de Login, registro de usuarios y recuperación de credenciales.

home/

— Vista de inicio con accesos rápidos, métricas de carga y resumen personalizado del usuario.

layout/

— Header principal con selector de roles y Sidebar de navegación responsiva.

common/

— Modales de confirmación accesibles y componentes reutilizables de UI.

providers/

— Contenedor raíz de contextos globales de la aplicación (`Providers.tsx`).

context/

— Proveedores de Estado React: `AuthContext.tsx` (sesión y RBAC) y `TimerContext.tsx` (cronómetro).

lib/

— Servicios de acceso a datos (`supabase.ts`, `coursesService.ts`, `categoryRates.ts`, etc.) y utilidades (`utils.ts`).

types/

— Definición central de interfaces TypeScript (`Task`, `Course`, `User`, `PlantillaTareaCurso`, etc.).

## 1.2 Patrones de Diseño Arquitectónico Implementados

Capa de Servicios

Desacoplada

PATRÓN SERVICE LAYER

Las consultas a la base de datos están encapsuladas en archivos dentro de `/src/lib/` (ej. `coursesService.ts`, `facultadesService.ts`). Los componentes de la UI no construyen queries directas de Supabase a bajo nivel, facilitando pruebas y mantenimiento.

Gestión de Estado

Híbrido

CONTEXT API + LOCAL STATE

Se adopta Context API para estados transversales globales (Sesión de Autenticación, Permisos del Perfil, y Cronómetro Flotante), mientras que el estado local efímero y de formularios se mantiene dentro de cada componente con React Hooks estandarizados.

Diseño Primero (Design-First)

HYBRID UI SYSTEM

La interfaz de usuario implementa un sistema de diseño híbrido con Tailwind CSS y Framer Motion, empleando elevación de capas, paletas cromáticas armónicas (Indigo, Esmeralda, Ámbar), microinteracciones fluidas y retroalimentación táctil de confirmación.

API Routes Seguras para Admin

BFF / SERVERLESS

Las operaciones administrativas críticas (como el aprovisionamiento de cuentas en Supabase Auth o el borrado forzado de perfiles) se ejecutan mediante Serverless API Routes en `/src/app/api/admin/` utilizando la clave maestra `SUPABASE_SERVICE_ROLE_KEY`, aislando el cliente de privilegios elevados.

## 2 Matriz de Dependencias y Tecnologías

El proyecto cuenta con un stack moderno, equilibrado y de alto rendimiento. Todas las librerías se encuentran sincronizadas y compatibles con React 18 y el motor de compilación Next.js 14.

| CAPA / CATEGORÍA         | LIBRERÍA & VERSIÓN           | PROPÓSITO PRINCIPAL                                                      | EVALUACIÓN ARQUITECTÓNICA                                         |
|--------------------------|------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|
| Framework Core           | next@14.2.15                 | Renderizado híbrido, enrutador App Router y Serverless Functions.        | <b>EXCELENTE</b> — Soporta SSR, RSC y optimización de assets.     |
| Librería Base            | react@18.3.1 / react-dom     | Biblioteca de construcción de interfaces basadas en componentes.         | <b>ESTABLE</b> — Máxima compatibilidad con el ecosistema.         |
| Tipado Estricto          | typescript@5.6.3             | Seguridad de tipos estática en tiempo de desarrollo y compilación.       | <b>ROBUSTO</b> — Evita errores en tiempo de ejecución.            |
| Framework CSS            | tailwindcss@3.4.14           | Sistema de diseño utilitario, responsivo y extensible.                   | <b>ÓPTIMO</b> — Cero CSS sobrante en producción.                  |
| Cliente Backend          | @supabase/supabase-js@2.45.0 | Conexión con PostgreSQL, Autenticación, RLS y RPCs.                      | <b>OFICIAL</b> — Manejo fluido de sesiones y DDL.                 |
| Iconografía              | lucide-react@0.453.0         | Set de más de 400 iconos SVG coherentes y optimizados para tree-shaking. | <b>LÍDER</b> — Reemplazo moderno y liviano a Feather Icons.       |
| Visualización de Datos   | recharts@2.13.0              | Gráficas de barras, dona, líneas y distribución de productividad.        | <b>POTENTE</b> — Gráficos declarativos e interactivos con SVG.    |
| Animaciones & UX         | framer-motion@11.11.8        | Transiciones entre pestañas, animaciones modales y microinteracciones.   | <b>FLUIDO</b> — Aumenta sustancialmente el valor percibido de UI. |
| Celebraciones & Feedback | canvas-confetti@1.9.4        | Efecto visual de confeti al completar tareas o cursos clave.             | <b>LIGERO</b> — Impacto positivo en la gamificación docente.      |

## 2.1 Gestión de Estado y Flujo de Datos

En lugar de incorporar gestores pesados y sobre-dimensionados (como Redux o Zustand), PrismaLab implementa **Context API** respaldada por estados locales efímeros. Para la sincronización con Supabase se utilizan llamadas asíncronas directas con manejo de estados de carga (loading) y banderas de error (error).

### **i** Nota de Arquitectura sobre Drag & Drop

El tablero Kanban implementa la **HTML5 Drag and Drop API nativa** con gestión de eventos (`onDragStart`, `onDragOver`, `onDrop`), eliminando la necesidad de paquetes pesados de terceros como `react-beautiful-dnd` o `@dnd-kit`, logrando un bundle de cliente significativamente más ligero.

### 3 Modelo de Datos y Capa Backend (Supabase PostgreSQL)

La base de datos de PrismaLab está diseñada bajo un modelo relacional de alta integridad en PostgreSQL (alojado en Supabase), estructurado a lo largo de **12 Fases de desarrollo DDL** con extensiones UUID nativas.

#### 3.1 Inventario de Entidades y Tablas Principales

| TABLA                         | CAMPOS CLAVE                                                                                                                                                 | DESCRIPCIÓN FUNCIONAL                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| usuarios                      | id (UUID PK → auth.users),<br>nombre_completo, email, rol_id,<br>activo, ultima_conexion,<br>firma_digital                                                   | Perfil ampliado de usuario<br>enlazado a la autenticación oficial<br>de Supabase.              |
| areas                         | id, nombre, nivel (1 a 6),<br>parent_id (Self FK), jefe_id,<br>color, icono                                                                                  | Estructura jerárquica de la<br>institución (Admin, CMU,<br>Departamentos, etc.).               |
| roles / roles_permisos        | id, nombre, area_id / rol_id,<br>permiso_id                                                                                                                  | Matriz de Control de Acceso<br>Basado en Roles (RBAC) con<br>permisos granulares.              |
| facultades                    | id, nombre, decano_id (FK<br>usuarios), color, icono                                                                                                         | Unidades académicas mayores de<br>la Universidad.                                              |
| programas                     | id, nombre, facultad_id (FK),<br>coordinador_id (FK usuarios)                                                                                                | Programas académicos adscritos<br>a cada facultad.                                             |
| cursos                        | id, nombre, codigo (UNIQUE),<br>programa_id, periodo, docente_id,<br>evaluador_id, estado                                                                    | Cursos virtuales que atraviesan<br>las fases de diseño, producción y<br>revisión.              |
| proyectos                     | id, nombre, descripcion, area_id,<br>lider_id, estado                                                                                                        | Proyectos especiales e iniciativas<br>institucionales transversales del<br>CCV.                |
| tareas                        | id, titulo, curso_id,<br>proyecto_id, responsable_id,<br>orden_tarea, estado,<br>estado_bloqueo,<br>dependencias_operativas (UUID[]),<br>plantilla_origen_id | Unidad operativa de trabajo.<br>Soporta time-tracking, tarifas y<br>dependencias secuenciales. |
| plantilla_tareas_curso        | id, titulo, descripcion, orden,<br>tipo_responsable,<br>tiempo_estimado, activa                                                                              | Catálogo maestro de tareas<br>predeterminadas para los cursos<br>virtuales.                    |
| plantilla_tareas_dependencias | id, tarea_plantilla_id,<br>depende_de_id                                                                                                                     | Grafo de dependencias que<br>define qué tareas bloquean a<br>cuáles en el pipeline.            |

| TABLA                  | CAMPOS CLAVE                                                                 | DESCRIPCIÓN FUNCIONAL                                                       |
|------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| parrilla_publicaciones | id, titulo, formato, canal, fecha_publicacion, estado, copy, link_arte       | Calendario editorial y gestión de publicaciones de marketing y divulgación. |
| registro_horas         | id, tarea_id, usuario_id, rol_destino, horas_registradas, fecha, descripcion | Registro de horas de productividad y cálculo de costos operativos.          |
| category_rates         | id, categoria, tarifa_hora                                                   | Tabulador salarial y de honorarios por categoría profesional CCV.           |

### 3.2 Políticas de Seguridad Row Level Security (RLS) y Jerarquía

La seguridad se basa en la jerarquía descendente de 6 niveles institucional:

- **Nivel 6 (Administrador General):** Acceso global e irrestricto mediante bypass seguro validado con la función `public.es_admin(auth.uid())`.
- **Nivel 5 (CMU / Centro de Medios):** Supervisión de producción audiovisual, diseño y contenidos en toda la universidad.
- **Nivel 4 (Departamentos / Jefaturas):** Visibilidad de su área y subáreas descendientes resuelta mediante la función recursiva `public.get_subarea_ids(root_area_id)`.
- **Nivel 3 (Decanaturas):** Supervisión de todas las facultades y programas asociados a su decano.
- **Nivel 2 (Coordinaciones de Programa):** Gestión de los cursos y docentes de su programa académico.
- **Nivel 1 (Docentes y Operativos):** Acceso restringido exclusivamente a las tareas y cursos donde han sido asignados como responsables directos.

 **Prevención de Recursión Infinita en Supabase RLS**

Para evitar el fallo habitual de recursión cíclica al evaluar si un usuario es administrador en las políticas de RLS, el esquema implementa la función `public.es_admin(user_id)` marcada como `SECURITY DEFINER STABLE`. Esto permite validar el rol del usuario de forma aislada sin re-disparar las políticas de la tabla usuarios.

### 3.3 Lógica de Negocio en Base de Datos: Triggers y Funciones RPC

**RPC inicializar\_tareas\_curso**

PIPELINE INSTANTIATION

Clona en bloque la secuencia predeterminada al crear un curso. Resuelve dinámicamente el responsable según su rol (Docente, Par Evaluador, Coordinador de Programa, Decano o CMU Fijo) y mapea el grafo de dependencias asignando DISPONIBLE a las iniciales y BLOQUEADA a las subsecuentes.

**Trigger trg\_desbloqueo\_cascada**

CASCADING UNBLOCKER

Al completarse una tarea, evalúa automáticamente todas las tareas dependientes del mismo curso. Si ya no tienen dependencias pendientes, las promueve a DISPONIBLE. Implementa además **Rollback Automático** si una tarea completada es devuelta a un estado anterior.

**RPC forzar\_desbloqueo\_admin**

EMERGENCY BYPASS

Permite a un usuario de Nivel 6 forzar el desbloqueo de cualquier tarea estancada por contingencia, registrando una traza de auditoría inmutable en la tabla de comentarios de la tarea.

**Trigger handle\_new\_user**

AUTH PROVISIONING

Escucha las inserciones en `auth.users` y aprovisiona automáticamente el registro correspondiente en la tabla `public.usuarios` con el rol asignado o Docente por defecto.

## 4 Inventario de Funcionalidades Técnicas y Módulos Activos

### 4.1 Sistema de Autenticación y Control de Acceso RBAC

El módulo de autenticación (`src/components/auth/LoginScreen.tsx` y `src/context/AuthContext.tsx`) gestiona el ciclo de vida completo de la sesión. Al autenticarse, se cargan los permisos específicos del perfil a través del RPC `get_mis_permisos()`. El sistema dispone de un **Simulador de Roles** en el Header (exclusivo para administradores), permitiendo verificar la experiencia de usuario bajo cualquier rol con un solo clic.

### 4.2 Estructura Jerárquica e Institucional

El panel de administración (`AdminDashboard.tsx`) se encuentra organizado en una navegación de dos niveles con 3 dominios funcionales:

- **Control de Acceso & RBAC:** Gestión integral de usuarios (activación/desactivación, roles asignados), matriz de permisos y árbol de áreas jerárquicas con asignación de líderes de departamento.
- **Gestión Académica & Cursos:** Administración de Facultades (con asignación de Decanos), Programas (con asignación de Coordinadores), Proyectos Especiales y el *Catálogo Maestro de Tareas de Plantilla*.
- **Operaciones & Servicios CCV:** Tabulador de tarifas por categoría profesional, bandeja de solicitudes de soporte y directorio operativo CCV.

### 4.3 Módulo de Tareas, Dependencias y Productividad

Las tareas se gestionan mediante `TaskDetailModal.tsx` y `TaskCard.tsx`. Este módulo integra:

- **Visualización de Bloqueo:** Insignias dinámicas que alertan si la tarea está BLOQUEADA por una tarea predecesora, indicando el título de la dependencia pendiente.
- **Cálculo Automático de Costos:** Integración con la tabla de tarifas por categoría para estimar el costo financiero de la tarea en función de las horas estimadas e invertidas.
- **Cronómetro Global en Vivo:** Controlado desde `GlobalTimerBar.tsx` y `TimerContext.tsx`, permitiendo a los operativos iniciar, pausar y registrar tiempos reales directamente en la tarea activa.

### 4.4 Vistas Avanzadas del Flujo de Trabajo

#### Tablero Kanban Dinámico

KANBANBOARD.TSX

Organizado en 4 columnas (*Pendiente, En Proceso, En Revisión, Completada*). Al arrastrar una tarjeta a 'Completada', se dispara automáticamente el trigger de desbloqueo en cascada y se disparan efectos visuales de celebración con `canvas-confetti`.

#### Calendario de Trabajo

CALENDARVIEW.TSX

Proyección cronológica mensual y semanal con diferenciación cromática por estado y facultad. Al pulsar sobre cualquier evento se abre de inmediato el modal de detalle con todas las acciones operativas disponibles.

#### Parrilla de Publicaciones

PARRILLAPUBLICACIONESVIEW.TSX

Planificador de contenidos multicanal (Instagram, LinkedIn, YouTube, Web CCV) con seguimiento de copias, enlaces a artes en Google Drive o Figma y estados de aprobación.

#### Monitor de Cursos y Proyectos

COURSEPROJECTPROGRESSMODAL.TSX

Visualizador de avance porcentual en tiempo real de cursos virtuales, mostrando la secuencia completa de tareas, dependencias completadas y cuellos de botella operativos.

## 4.5 Panel Analítico y Dashboard

El módulo `DashboardAnalytics.tsx` sintetiza métricas clave mediante tarjetas de KPI (Total de Cursos Activos, Tareas en Proceso, Horas Invertidas, Eficiencia de Cumplimiento) y gráficos interactivos de **Recharts** que desglosan la productividad por área, docente y estado de avance.

## 4.6 Canales de Integración Externa Preparados

- **Sincronización con ClickUp:** La tabla `tareas` cuenta con el campo `clickup_task_id` para enlace bidireccional de tareas externas.
- **Webhooks de Automatización (Make / n8n):** Estructura lista en backend para emisión de payloads JSON al aprovisionar nuevos cursos virtuales.
- **Estructura Parametrizada de Google Drive:** Enlace nativo mediante el campo `enlace_recurso` en tareas y parrilla de publicaciones.
- **Firma Digital Táctil:** Soporte en el esquema (`usuarios.firma_digital`) para validación de actas y entregables académicos.

## 5 Variables de Entorno y Conexiones Externas

La configuración de conectividad del proyecto se gestiona mediante variables de entorno estandarizadas de Next.js, separando las credenciales públicas accesibles en el navegador de las credenciales con privilegios elevados del servidor.

| VARIABLE DE ENTORNO           | ÁMBITO / SCOPE         | SERVICIO DESTINO     | PROPÓSITO Y NIVEL DE SENSIBILIDAD                                                                                                               |
|-------------------------------|------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| NEXT_PUBLIC_SUPABASE_URL      | CLIENTE & SERVIDOR     | Supabase API Gateway | URL del proyecto Supabase (ej. <code>https://xyz.supabase.co</code> ). Público, define el host de la base de datos y endpoints REST.            |
| NEXT_PUBLIC_SUPABASE_ANON_KEY | CLIENTE & SERVIDOR     | Supabase Auth / REST | Clave pública anónima. Protegida estrictamente por las políticas de Row Level Security (RLS) en la base de datos.                               |
| SUPABASE_SERVICE_ROLE_KEY     | SOLO SERVIDOR (SECRET) | Supabase Auth Admin  | Clave maestra con bypass de RLS. Utilizada exclusivamente en rutas <code>/api/admin/*</code> para gestión administrativa de cuentas de usuario. |
| NEXT_PUBLIC_MAKE_WEBHOOK_URL  | OPCIONAL               | Make / n8n Workflow  | Endpoint webhook para automatizaciones externas al instanciar cursos o notificar eventos críticos.                                              |

⚠ Regla Crítica de Seguridad: Service Role Key

Bajo ninguna circunstancia se debe anteponer el prefijo `NEXT_PUBLIC_` a la variable `SUPABASE_SERVICE_ROLE_KEY`. Esta clave debe residir exclusivamente en el entorno de ejecución del servidor (Vercel / Node.js) para impedir que sea expuesta en el bundle de JavaScript del cliente.

## 6 Deuda Técnica, Puntos Críticos y Recomendaciones

### 6.1 Hallazgos de la Auditoría

Consultas en Cascada sin Caché

RENDIMIENTO

Actualmente, varios componentes ejecutan llamadas directas a Supabase en sus efectos de montaje sin un mecanismo de deduplicación de peticiones (como TanStack Query o SWR). Si múltiples modales se abren sucesivamente, se repiten consultas a tablas maestras (facultades, programas, áreas).

Índices Faltantes en Dependencias

BASE DE DATOS

El campo `tareas.dependencias_operativas` almacena un array `UUID[]`. En cursos con secuencias de más de 30 tareas, la consulta `NEW.id = ANY(dependencias_operativas)` en el trigger de cascada puede degradarse sin un índice de tipo **GIN** sobre este array.

Suscripción a Realtime Supabase

ESCALABILIDAD

Validación de Formularios

MANTENIBILIDAD

El tablero Kanban actualiza estados de forma reactiva local, pero no escucha los canales de *Supabase Realtime WebSockets*. Si dos administradores trabajan simultáneamente, uno no verá de inmediato el cambio de estado del otro sin recargar la vista.

Los formularios de creación de cursos y tareas manejan validación manual. Se recomienda estandarizar con zod y react-hook-form para unificar mensajes de error y esquemas de validación compartidos entre cliente y servidor.

## 6.2 Recomendaciones Inmediatas para Despliegue a Producción

1. **Crear Índice GIN en Postgres:** Ejecutar en el SQL Editor de Supabase:

```
CREATE INDEX IF NOT EXISTS idx_tareas_deps_gin ON public.tareas USING GIN
(dependencias_operativas);
```

2. **Implementar TanStack Query (React Query):** Configurar un QueryClient con un staleTime de 5 minutos para tablas paramétricas (áreas, facultades, programas y roles).
3. **Habilitar Supabase Realtime en Tareas:** Activar la replicación en la tabla tareas para permitir sincronización colaborativa multi-usuario en el Tablero Kanban.
4. **Variables de Entorno en Vercel:** Asegurarse de cargar NEXT\_PUBLIC\_SUPABASE\_URL, NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY y SUPABASE\_SERVICE\_ROLE\_KEY en el panel de Vercel (Production & Preview).
5. **Población de Datos Semilla:** Ejecutar el script supabase/seed\_plantilla\_cursos.sql en el entorno de producción para cargar el catálogo inicial de tareas y dependencias predeterminadas.